

Phase 4 Structured Notes: Queue Implementation in C++ Using OOP

1. Current Progress Before Phase 4

Before Phase 4, you already learned the basic queue idea and how to build a queue using a C-style program.

In **Phase 1**, you learned that a queue follows this rule:

FIFO

FIFO means:

First In, First Out

That means the first value inserted into the queue is the first value removed from the queue.

You also learned these basic queue operations:

Operation	Simple Meaning
<code>enqueue()</code>	Insert a value into the queue
<code>dequeue()</code>	Remove a value from the queue
<code>front</code>	The side where deletion happens
<code>rear</code>	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In **Phase 2**, you learned how to store a queue inside an array.

You compared two designs:

Design	Main Idea	Result
Single-pointer queue	Uses only <code>rear</code>	Deletion is slow because elements must shift
Two-pointer queue	Uses <code>front</code> and <code>rear</code>	Deletion is fast because only <code>front</code> moves

The most important Phase 2 lesson was:

The two-pointer queue avoids shifting.

In **Phase 3**, you built a complete linear array queue in C using:

- `struct`
- `malloc()`
- `free()`
- separate functions
- `front` and `rear`
- `create()`
- `enqueue()`
- `dequeue()`
- `display()`

The Phase 3 queue used this structure:

```
struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};
```

And you called functions like this:

```
enqueue(&q, 10);
dequeue(&q);
display(q);
```

Now Phase 4 changes the style.

Instead of using a C `struct` and separate functions, Phase 4 uses a C++ `class`.

The queue logic stays mostly the same, but the organization becomes cleaner.

2. Main Goal of Phase 4

The goal of Phase 4 is to implement the same queue using **C++ Object-Oriented Programming**, also called **OOP**.

In simple words:

```
Phase 3: Queue using C struct and separate functions.  
Phase 4: Queue using C++ class and object methods.
```

The queue still follows FIFO.

The queue still inserts at the rear.

The queue still deletes from the front.

But now the data and functions are kept together inside one class.

3. What Is OOP?

OOP means:

```
Object-Oriented Programming
```

In OOP, we create objects.

An object is like a small package that contains:

1. Data
2. Functions that work on that data

For a queue object, the data includes:

```
size  
front  
rear  
Q
```

The functions include:

```
enqueue()  
dequeue()  
display()
```

So instead of keeping data and functions separate, OOP keeps them together.

4. Phase 3 Style vs Phase 4 Style

In Phase 3, the queue was used like this:

```
struct Queue q;  
create(&q, 5);  
enqueue(&q, 10);  
dequeue(&q);  
display(q);
```

In Phase 4, the queue is used like this:

```
Queue q(5);  
q.enqueue(10);  
q.dequeue();  
q.display();
```

The Phase 4 style is cleaner because the queue object manages itself.

Phase 3 C Style	Phase 4 C++ OOP Style
Uses <code>struct Queue</code>	Uses <code>class Queue</code>
Uses separate functions	Uses member functions inside the class
Calls <code>create(&q, 5)</code>	Constructor runs automatically
Calls <code>enqueue(&q, 10)</code>	Calls <code>q.enqueue(10)</code>
Data may be directly accessed	Data is protected using <code>private</code>
Uses <code>malloc()</code> and <code>free()</code>	Uses <code>new[]</code> and <code>delete[]</code>

5. What Is a Class?

A **class** is a blueprint for creating objects.

A class tells C++:

This object will have these variables.
This object will have these functions.

For a queue, the class can look like this:

```
class Queue {  
private:  
    int size;  
    int front;  
    int rear;  
    int *Q;  
  
public:  
    Queue();  
    Queue(int size);  
    void enqueue(int x);  
    int dequeue();  
    void display();  
};
```

This class says:

- A queue has a `size`.
- A queue has a `front` index.
- A queue has a `rear` index.
- A queue has an array pointer `Q`.
- A queue can perform `enqueue()`.
- A queue can perform `dequeue()`.
- A queue can perform `display()`.

6. What Is an Object?

An object is a real variable created from a class.

Example:

```
Queue q(5);
```

Here:

Queue

is the class name.

q

is the object name.

The object `q` is an actual queue that can store values.

You can call its functions like this:

```
q.enqueue(10);  
q.display();
```

This means:

Use the queue object `q` to insert 10.
Use the queue object `q` to display its values.

7. Data Members

Variables inside a class are called **data members**.

In this queue class, the data members are:

```
int size;  
int front;  
int rear;  
int *Q;
```

Their meanings are:

Data Member	Simple Meaning
size	Maximum number of values the queue can store
front	Index used for deletion
rear	Index used for insertion

Data Member	Simple Meaning
<code>Q</code>	Pointer to the array that stores queue values

Important idea:

front and rear are index values.
They are not memory addresses.

Example:

```
front = 0;
rear = 2;
```

This means `front` and `rear` are positions inside the array.

8. Member Functions

Functions inside a class are called **member functions** or **methods**.

In this queue class, the member functions are:

```
Queue();
Queue(int size);
void enqueue(int x);
int dequeue();
void display();
```

Their meanings are:

Member Function	Simple Meaning
<code>Queue()</code>	Default constructor
<code>Queue(int size)</code>	Parameterized constructor
<code>enqueue(int x)</code>	Insert value <code>x</code> at the rear
<code>dequeue()</code>	Delete and return the front value
<code>display()</code>	Print all valid queue values

9. Private and Public Sections

A C++ class usually has two important sections:

```
private:
```

and

```
public:
```

Private Section

The `private` section contains data that should be protected.

Example:

```
private:
    int size;
    int front;
    int rear;
    int *Q;
```

Private members cannot be directly accessed from outside the class.

That means outside code cannot do this:

```
q.front = 100;
q.rear = -50;
```

This is good because it prevents the queue from being damaged.

Public Section

The `public` section contains functions that outside code is allowed to call.

Example:

```
public:
    void enqueue(int x);
```



```
int dequeue();  
void display();
```

Outside code can do this:

```
q.enqueue(10);  
q.dequeue();  
q.display();
```

So the public functions are the safe way to use the queue.

10. Encapsulation

Encapsulation means:

Keep data protected inside the class and allow access only through safe public functions.

In this queue, encapsulation protects:

```
size  
front  
rear  
Q
```

Why is this useful?

Because outside code should not directly change `front` or `rear`.

If outside code changes `front` or `rear` incorrectly, the queue may break.

Example of dangerous code:

```
q.rear = 100;
```

If the array size is only 5, then `rear = 100` is invalid.

The program may try to access memory outside the array.

That is why `front`, `rear`, `size`, and `Q` should be private.

The user of the queue should only use:

```
q.enqueue(10);  
q.dequeue();  
q.display();
```

This keeps the queue safe.

11. Basic Queue Class Design

A beginner-friendly queue class looks like this:

```
class Queue {  
private:  
    int size;  
    int front;  
    int rear;  
    int *Q;  
  
public:  
    Queue();  
    Queue(int size);  
    void enqueue(int x);  
    int dequeue();  
    void display();  
};
```

This is only the class design.

It shows what the class contains.

The actual function bodies can be written inside the class or outside the class.

12. Constructor

A **constructor** is a special function that runs automatically when an object is created.

A constructor has the same name as the class.

For this class:

```
class Queue
```

The constructor is named:

```
Queue()
```

A constructor does not have a return type.

Wrong:

```
void Queue() // wrong
```

Correct:

```
Queue() // correct
```

The constructor prepares the object before we use it.

For a queue, the constructor should:

1. Set the size.
2. Set `front` and `rear`.
3. Create the array.

13. Default Constructor

A **default constructor** creates an object without needing extra information.

Example:

```
Queue q;
```

This creates a queue using the default constructor.

Default constructor code:

```
Queue() {  
    size = 10;
```

```
    front = rear = -1;  
    Q = new int[size];  
}
```

Line by line:

```
size = 10;
```

This gives the queue a default capacity of 10.

```
front = rear = -1;
```

This makes the queue empty.

```
Q = new int[size];
```

This creates an array in heap memory.

After this constructor runs:

```
size = 10  
front = -1  
rear = -1  
Q = array of 10 integers
```

The queue is ready to use.

14. Parameterized Constructor

A **parameterized constructor** lets the user choose the queue size.

Example:

```
Queue q(5);
```

This creates a queue with size 5.

Parameterized constructor code:

```
Queue(int size) {  
    this->size = size;  
    front = rear = -1;  
    Q = new int[this->size];  
}
```

This constructor receives a value called `size`.

If we write:

```
Queue q(5);
```

then the parameter `size` receives the value `5`.

15. Meaning of `this->`

Inside the parameterized constructor, we write:

```
this->size = size;
```

This line can be confusing for beginners because the word `size` appears twice.

The left side:

```
this->size
```

means:

```
The size variable that belongs to this object.
```

The right side:

```
size
```

means:

The parameter passed into the constructor.

So this line means:

Store the parameter size inside the object's size variable.

Example:

```
Queue q(5);
```

Then:

```
this->size = size;
```

means:

```
q.size = 5
```

We use `this->` when the parameter name is the same as the data member name.

16. Dynamic Memory in C++

In Phase 3, you used `malloc()` to create memory in C.

In Phase 4, we use `new[]` in C++.

Example:

```
Q = new int[size];
```

This creates an integer array in heap memory.

If `size = 5`, then this line creates space for 5 integers.

The pointer `Q` points to that array.

Simple meaning:

Create an array for the queue while the program is running.

17. Destructor

A **destructor** is a special function that runs automatically when the object is destroyed.

A destructor has the same name as the class, but it starts with `~`.

For the `Queue` class, the destructor is:

```
~Queue()
```

Destructor code:

```
~Queue() {  
    delete[] Q;  
}
```

The constructor creates memory using:

```
new int[size]
```

So the destructor releases that memory using:

```
delete[] Q;
```

Simple meaning:

Constructor creates the array.
Destructor cleans up the array.

This prevents memory leaks.

18. Why Use `delete[]`?

If we create an array using:

```
new int[size]
```

we must release it using:

```
delete[] Q;
```

The square brackets are important because `Q` points to an array, not just one integer.

Wrong:

```
delete Q;           // wrong for arrays
```

Correct:

```
delete[] Q;        // correct for arrays
```

19. Scope Resolution Operator `::`

The scope resolution operator is:

```
::
```

It tells C++ that a function belongs to a class.

For example, if we write a function outside the class, we can write:

```
void Queue::enqueue(int x) {  
    // function body  
}
```

This means:

```
enqueue() belongs to the Queue class.
```

Without `Queue::`, C++ may think `enqueue()` is just a normal function, not a class function.

Example:


```

class Queue {
private:
    int size;
    int front;
    int rear;
    int *Q;

public:
    Queue(int size);
    void enqueue(int x);
};

Queue::Queue(int size) {
    this->size = size;
    front = rear = -1;
    Q = new int[this->size];
}

void Queue::enqueue(int x) {
    if (rear == size - 1) {
        cout << "Queue Full\n";
    } else {
        rear++;
        Q[rear] = x;
    }
}

```

The `Queue::` part connects the function to the `Queue` class.

20. Function Definitions Inside the Class

One way to write the queue class is to put function bodies directly inside the class.

Example:

```

class Queue {
private:
    int size;
    int front;
    int rear;
    int *Q;

public:
    Queue(int size) {

```

```

        this->size = size;
        front = rear = -1;
        Q = new int[this->size];
    }

    void enqueue(int x) {
        if (rear == size - 1) {
            cout << "Queue Full\n";
        } else {
            rear++;
            Q[rear] = x;
        }
    }
};

```

This is simple for beginners.

21. Function Definitions Outside the Class

Another way is to write only declarations inside the class, then define the functions outside using `Queue::`.

Example:

```

class Queue {
private:
    int size;
    int front;
    int rear;
    int *Q;

public:
    Queue(int size);
    void enqueue(int x);
};

```

Then outside the class:

```

Queue::Queue(int size) {
    this->size = size;
    front = rear = -1;
    Q = new int[this->size];
}

```

```
void Queue::enqueue(int x) {  
    if (rear == size - 1) {  
        cout << "Queue Full\n";  
    } else {  
        rear++;  
        Q[rear] = x;  
    }  
}
```

Both styles are correct.

For learning, writing functions inside the class is easier.

For larger programs, writing them outside the class can make the code cleaner.

22. Empty Condition

The empty condition is the same as Phase 3:

```
front == rear
```

This means the queue has no valid elements.

Example 1:

```
front = -1  
rear = -1
```

The queue is empty.

Example 2:

```
front = 2  
rear = 2
```

The queue is also empty.

Why?

Because valid elements are between:

```
front + 1
```

and

```
rear
```

If `front` and `rear` are equal, there is no valid element between them.

23. Full Condition

The full condition is also the same as Phase 3:

```
rear == size - 1
```

This is a linear array queue.

Array indexes start from 0.

If `size = 5`, the valid indexes are:

```
0 1 2 3 4
```

The last index is:

```
size - 1
```

So if:

```
rear = 4
```

then the queue is full.

24. Important Valid Range Rule

In this queue design:

```
front
```

does not point directly to the first valid element.

Instead:

```
front points before the first valid element.
```

So the first valid element is at:

```
front + 1
```

The last valid element is at:

```
rear
```

So the valid queue range is:

```
front + 1 to rear
```

Example:

```
front = 0  
rear = 2  
Array = [10, 20, 30]
```

The valid range is:

```
front + 1 = 1  
rear = 2
```

So the logical queue is:

```
20 30
```

The value `10` is ignored because `front` has moved past it.

25. Enqueue Function in C++

`enqueue()` inserts a value at the rear of the queue.

Code:

```
void enqueue(int x) {  
    if (rear == size - 1) {  
        cout << "Queue Full\n";  
    } else {  
        rear++;  
        Q[rear] = x;  
    }  
}
```

Logic of `enqueue()`

To insert a value:

1. Check if the queue is full.
2. If full, print a message.
3. If not full, move `rear` one step forward.
4. Store the value at `Q[rear]`.

Line-by-Line Explanation

Full check:

```
if (rear == size - 1)
```

This checks whether `rear` is already at the last array index.

Full message:

```
cout << "Queue Full\n";
```

This prints a warning if there is no more space.

Move rear:

```
rear++;
```

This moves `rear` to the next position.

Store value:

```
Q[rear] = x;
```

This stores the new value at the rear position.

26. Enqueue Dry Run

Start:

```
size = 5  
front = -1  
rear = -1  
Queue is empty
```

Call:

```
q.enqueue(10);
```

Step 1: Check full.

```
rear == size - 1  
-1 == 4
```

False. The queue is not full.

Step 2: Move `rear`.

```
rear moves from -1 to 0
```

Step 3: Store value.

```
Q[0] = 10
```

Now:

```
front = -1  
rear = 0  
Queue = 10
```

Call:

```
q.enqueue(20);
```

Now:

```
front = -1  
rear = 1  
Queue = 10 20
```

Call:

```
q.enqueue(30);
```

Now:

```
front = -1  
rear = 2  
Queue = 10 20 30
```

27. Time Complexity of Enqueue

`enqueue()` does not use a loop.

It only:

- checks if the queue is full
- moves `rear`
- stores one value

So the time complexity is:

```
O(1)
```


`O(1)` means constant time.

The operation does not become slower when the queue has more elements.

28. Dequeue Function in C++

`dequeue()` removes and returns the front value.

Code:

```
int dequeue() {
    int x = -1;

    if (front == rear) {
        cout << "Queue Empty\n";
    } else {
        front++;
        x = Q[front];
    }

    return x;
}
```

Logic of `dequeue()`

To remove a value:

1. Create a variable `x` and set it to `-1`.
 2. Check if the queue is empty.
 3. If empty, print a message.
 4. If not empty, move `front` one step forward.
 5. Store the deleted value in `x`.
 6. Return `x`.
-

29. Why `x` Starts as `-1`

At the beginning of `dequeue()`, we write:

```
int x = -1;
```

This means:

```
If deletion fails, return -1.
```

For example, if the queue is empty, there is no real value to delete.

So the function returns `-1` as a signal.

30. Dequeue Line-by-Line Explanation

Create default value:

```
int x = -1;
```

This prepares a value to return if deletion fails.

Check empty:

```
if (front == rear)
```

This checks if the queue has no valid elements.

Empty message:

```
cout << "Queue Empty\n";
```

This prints a warning if there is nothing to delete.

Move front:

```
front++;
```

This moves `front` to the next valid element.

Store deleted value:

```
x = Q[front];
```

This stores the value being removed.

Return deleted value:

```
return x;
```

This sends the deleted value back to the caller.

31. Dequeue Dry Run

Suppose the queue contains:

```
10 20 30
```

Current values:

```
front = -1  
rear = 2
```

Array:

```
Index:  0   1   2  
Array: 10  20  30
```

Call:

```
q.dequeue();
```

Step 1: Check empty.

```
front == rear  
-1 == 2
```

False. The queue is not empty.

Step 2: Move `front`.

```
front moves from -1 to 0
```

Step 3: Get value.

```
x = Q[0]
x = 10
```

So 10 is returned.

Now:

```
front = 0
rear = 2
```

The logical queue is:

```
20 30
```

32. Logical Deletion

In this array queue, deletion is logical.

That means the old value may still physically exist in the array, but it is no longer part of the queue.

Example before deletion:

```
front = -1
rear = 2
Array = [10, 20, 30]
```

After one deletion:

```
front = 0
rear = 2
Array may still be [10, 20, 30]
```

But the logical queue is:

```
20 30
```

Why?

Because the valid range is now:

```
front + 1 to rear
```

That means:

```
1 to 2
```

So the valid values are:

```
Q[1] = 20  
Q[2] = 30
```

The old value `10` is ignored.

33. Time Complexity of Dequeue

`dequeue()` does not shift elements.

It only:

- checks if the queue is empty
- moves `front`
- returns one value

So the time complexity is:

```
O(1)
```

This is fast.

34. Display Function in C++

`display()` prints all valid queue values.

Code:

```
void display() {  
    for (int i = front + 1; i <= rear; i++) {  
        cout << Q[i] << " ";  
    }  
    cout << endl;  
}
```

35. Why Display Starts at `front + 1`

In this design:

`front`

does not directly point to the first valid element.

It points before the first valid element.

So the first valid element is:

`front + 1`

That is why display starts here:

```
int i = front + 1
```

36. Why Display Stops at `rear`

`rear` points to the last inserted value.

So `rear` is the last valid element.

That is why the loop condition is:

`i <= rear`

The display range is:

```
front + 1 to rear
```

37. Display Dry Run

Suppose:

```
front = 0  
rear = 2  
Array = [10, 20, 30]
```

The display loop starts at:

```
front + 1 = 1
```

The display loop stops at:

```
rear = 2
```

So it prints:

```
Q[1] Q[2]
```

Output:

```
20 30
```

The value `10` is not printed because it was already removed logically.

38. Time Complexity of Display

`display()` prints every valid element.

If there are `n` valid elements, it prints `n` values.

So the time complexity is:

$O(n)$

39. Complete Phase 4 Program

```
#include <iostream>
using namespace std;

class Queue {
private:
    int size;
    int front;
    int rear;
    int *Q;

public:
    Queue() {
        size = 10;
        front = rear = -1;
        Q = new int[size];
    }

    Queue(int size) {
        this->size = size;
        front = rear = -1;
        Q = new int[this->size];
    }

    ~Queue() {
        delete[] Q;
    }

    void enqueue(int x) {
        if (rear == size - 1) {
            cout << "Queue Full\n";
        } else {
            rear++;
            Q[rear] = x;
        }
    }

    int dequeue() {
        int x = -1;

        if (front == rear) {
```



```

        cout << "Queue Empty\n";
    } else {
        front++;
        x = Q[front];
    }

    return x;
}

void display() {
    for (int i = front + 1; i <= rear; i++) {
        cout << Q[i] << " ";
    }
    cout << endl;
}

};

int main() {
    Queue q(5);

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    q.display();           // 10 20 30

    q.dequeue();           // removes 10

    q.display();           // 20 30

    return 0;
}

```

40. Complete Program Explanation

Header File

```
#include <iostream>
```

This allows us to use `cout` .

Namespace

```
using namespace std;
```

This allows us to write `cout` instead of `std::cout`.

Class Declaration

```
class Queue {
```

This begins the queue class.

Private Data

```
private:
    int size;
    int front;
    int rear;
    int *Q;
```

These variables are protected inside the class.

Public Functions

```
public:
```

The functions below this can be called from `main()`.

Default Constructor

```
Queue() {
    size = 10;
    front = rear = -1;
    Q = new int[size];
}
```

This creates a queue of size 10 if no size is given.

Parameterized Constructor

```
Queue(int size) {  
    this->size = size;  
    front = rear = -1;  
    Q = new int[this->size];  
}
```

This creates a queue using the size given by the user.

Destructor

```
~Queue() {  
    delete[] Q;  
}
```

This releases the array memory when the object is destroyed.

Main Function

```
int main() {  
    Queue q(5);
```

This creates a queue object named `q` with capacity 5.

Then:

```
q.enqueue(10);  
q.enqueue(20);  
q.enqueue(30);
```

This inserts 10, 20, and 30.

Then:

```
q.display();
```

This prints:

```
10 20 30
```

Then:

```
q.dequeue();
```

This removes 10.

Then:

```
q.display();
```

This prints:

```
20 30
```

41. Full Program Dry Run

Start

```
Queue q(5);
```

After this:

```
size = 5  
front = -1  
rear = -1  
Queue is empty
```

After `q.enqueue(10)`

```
front = -1  
rear = 0  
Q[0] = 10  
Logical queue = 10
```

After `q.enqueue(20)`

```
front = -1
rear = 1
Q[0] = 10
Q[1] = 20
Logical queue = 10 20
```

After `q.enqueue(30)`

```
front = -1
rear = 2
Q[0] = 10
Q[1] = 20
Q[2] = 30
Logical queue = 10 20 30
```

First Display

```
q.display();
```

Display starts at:

```
front + 1 = 0
```

Display stops at:

```
rear = 2
```

Output:

```
10 20 30
```

After `q.dequeue()`

Before deletion:

```
front = -1  
rear = 2
```

Move `front`:

```
front moves from -1 to 0
```

Return:

```
Q[0] = 10
```

Now:

```
front = 0  
rear = 2  
Logical queue = 20 30
```

Second Display

```
q.display();
```

Display starts at:

```
front + 1 = 1
```

Display stops at:

```
rear = 2
```

Output:

```
20 30
```

42. Expected Output

```
10 20 30
20 30
```

This output proves FIFO.

The values entered in this order:

```
10 20 30
```

The first removed value was:

```
10
```

So the first inserted value left first.

43. Visual Pointer Table

Step	Operation	front	rear	Logical Queue
1	Create Queue q(5)	-1	-1	Empty
2	q.enqueue(10)	-1	0	10
3	q.enqueue(20)	-1	1	10, 20
4	q.enqueue(30)	-1	2	10, 20, 30
5	q.dequeue()	0	2	20, 30

Important observation:

After `dequeue()`, `front` becomes 0.

That means index 0 is no longer part of the logical queue.

44. Object Syntax

In C++ OOP, we use object syntax:

```
q.enqueue(10);  
q.dequeue();  
q.display();
```

The dot operator `.` means:

Call this function using this object.

Example:

```
q.enqueue(10);
```

means:

Use object `q` to call `enqueue` and insert 10.

This is cleaner than the C style:

```
enqueue(&q, 10);
```

45. Why We No Longer Pass `&q`

In Phase 3, you wrote:

```
enqueue(&q, 10);
```

because `enqueue()` was a separate function and needed the address of the queue.

In Phase 4, you write:

```
q.enqueue(10);
```

because `enqueue()` belongs to the object `q`.

The function already knows which queue object it is working on.

So we do not need to pass `&q`.

46. Complexity Summary

Operation	Time Complexity	Why?
Constructor	<code>O(1)</code>	Sets values and creates array
<code>enqueue()</code>	<code>O(1)</code>	Moves <code>rear</code> and stores one value
<code>dequeue()</code>	<code>O(1)</code>	Moves <code>front</code> and returns one value
<code>display()</code>	<code>O(n)</code>	Prints all valid elements
Destructor	<code>O(1)</code>	Releases the dynamic array

Space complexity:

```
O(size)
```

because the queue uses an array of the chosen size.

47. Overflow in Phase 4

Overflow happens when we try to insert into a full queue.

The full condition is:

```
rear == size - 1
```

Example:

```
size = 5
rear = 4
```

The queue is full because index 4 is the last valid index.

If we try to insert another value, the program prints:

```
Queue Full
```

48. Underflow in Phase 4

Underflow happens when we try to delete from an empty queue.

The empty condition is:

```
front == rear
```

Example:

```
front = -1  
rear = -1
```

The queue is empty.

If we try to delete, the program prints:

```
Queue Empty
```

and returns:

```
-1
```

49. C Style vs C++ OOP Style Comparison

Idea	C Style Queue	C++ OOP Queue
Main structure	<code>struct Queue</code>	<code>class Queue</code>
Data protection	Usually less protected	Data is private
Setup	<code>create(&q, size)</code>	Constructor runs automatically
Insert call	<code>enqueue(&q, x)</code>	<code>q.enqueue(x)</code>
Delete call	<code>dequeue(&q)</code>	<code>q.dequeue()</code>
Display call	<code>display(q)</code>	<code>q.display()</code>
Memory creation	<code>malloc()</code>	<code>new[]</code>
Memory release	<code>free()</code>	<code>delete[]</code> inside destructor

Idea	C Style Queue	C++ OOP Queue
Organization	Data and functions separate	Data and functions together

50. Why Phase 4 Is Cleaner

Phase 4 is cleaner because the queue object controls its own data.

In Phase 3, the program had to manually pass the queue address:

```
enqueue(&q, 10);
```

In Phase 4, the object handles the operation:

```
q.enqueue(10);
```

This is easier to read.

It also protects important variables using `private`.

So Phase 4 improves organization and safety.

51. Common Beginner Mistakes in Phase 4

Mistake 1: Making Data Public

Wrong:

```
public:  
    int front;  
    int rear;
```

Problem:

Outside code can damage the queue by changing `front` and `rear` directly.

Correct:

```
private:
    int front;
    int rear;
```

Mistake 2: Forgetting the Constructor

If the constructor does not create memory for `Q`, then the queue cannot safely store values.

Wrong idea:

```
Queue q;
q.enqueue(10); // dangerous if Q was never created
```

Correct idea:

The constructor should include:

```
Q = new int[size];
```

Mistake 3: Forgetting `this->`

When the parameter name and data member name are the same, use `this->`.

Correct:

```
Queue(int size) {
    this->size = size;
}
```

Here:

```
this->size
```

means the object's data member.

```
size
```

means the constructor parameter.

Mistake 4: Forgetting `delete[]`

If memory is created using:

```
new int[size]
```

then release it using:

```
delete[] Q;
```

This should usually be done in the destructor.

Mistake 5: Using `delete` Instead of `delete[]`

Wrong:

```
delete Q;
```

Correct:

```
delete[] Q;
```

Use `delete[]` because `Q` points to an array.

Mistake 6: Starting Display from `front`

Wrong:

```
for (int i = front; i <= rear; i++)
```

Correct:

```
for (int i = front + 1; i <= rear; i++)
```

Why?

Because `front` points before the first valid element.

Mistake 7: Thinking `front` Points Directly to the First Value

In this design, `front` does not point directly to the first valid value.

The first valid value is at:

```
front + 1
```

Mistake 8: Using the Wrong Empty Condition

Wrong:

```
front == -1
```

Correct:

```
front == rear
```

Why?

After some deletions, the queue can become empty when both `front` and `rear` are equal at another index.

Example:

```
front = 2  
rear = 2
```

This is empty.

Mistake 9: Forgetting the Full Condition

Correct full condition:

```
rear == size - 1
```

This is used in the simple linear array queue.

Mistake 10: Forgetting `Queue::` When Defining Functions Outside the Class

Wrong:

```
void enqueue(int x) {  
    // code  
}
```

This creates a normal function, not a class method.

Correct:

```
void Queue::enqueue(int x) {  
    // code  
}
```

`Queue::` tells C++ that the function belongs to the `Queue` class.

52. Important Terms from Phase 4

Term	Simple Meaning
OOP	Object-Oriented Programming
Class	Blueprint for creating objects
Object	Real variable created from a class
Data member	Variable inside a class
Member function	Function inside a class
Method	Another name for member function
Private	Can only be accessed inside the class
Public	Can be accessed from outside the class
Encapsulation	Protecting data and using public functions safely

Term	Simple Meaning
Constructor	Function that runs automatically when object is created
Default constructor	Constructor with no arguments
Parameterized constructor	Constructor with arguments
Destructor	Function that runs automatically when object is destroyed
<code>this</code>	Pointer to the current object
<code>this->size</code>	The <code>size</code> variable belonging to the current object
<code>new[]</code>	Creates a dynamic array in C++
<code>delete[]</code>	Releases a dynamic array in C++
<code>::</code>	Scope resolution operator
<code>q.enqueue(10)</code>	Object syntax for calling a method

53. Phase 4 Mini Review Questions

Question 1

What is the main goal of Phase 4?

Answer:

To implement the queue using a C++ class and OOP.

Question 2

Does Phase 4 change the FIFO rule?

Answer:

No. The queue still follows FIFO.

Question 3

What does FIFO mean?

Answer:

First In, First Out.

Question 4

What is a class?

Answer:

A class is a blueprint for creating objects.

Question 5

What is an object?

Answer:

An object is a real variable created from a class.

Question 6

Why are `front`, `rear`, `size`, and `Q` private?

Answer:

To protect them from being changed incorrectly by outside code.

Question 7

What is encapsulation?

Answer:

Encapsulation means keeping data protected inside a class and using public functions to access it safely.

Question 8

What does the constructor do?

Answer:

It prepares the queue object by setting values and creating the array.

Question 9

What does the destructor do?

Answer:

It releases the memory used by the queue array.

Question 10

What does `this->size = size;` mean?

Answer:

It stores the constructor parameter `size` into the object's `size` data member.

Question 11

What is the empty condition?

Answer:

```
front == rear
```

Question 12

What is the full condition?

Answer:

```
rear == size - 1
```

Question 13

Where is the first valid element?

Answer:

```
front + 1
```

Question 14

Where is the last valid element?

Answer:

```
rear
```

Question 15

What does `q.enqueue(10)` mean?

Answer:

It means call the `enqueue()` function using the queue object `q` and insert 10.

Question 16

What operator is used to define class functions outside the class?

Answer:

```
::
```

This is called the scope resolution operator.

54. Phase 4 Final Summary

In Phase 4, you learned how to implement a queue using C++ OOP.

The queue still follows:

FIFO: First In, First Out

The queue still uses:

```
front  
rear  
size  
Q
```

The queue still has these conditions:

```
front == rear    // empty  
rear == size - 1 // full  
front + 1        // first valid element  
rear             // last valid element
```

The main difference is organization.

In Phase 3, you used:

```
struct Queue
```

and separate functions.

In Phase 4, you use:

```
class Queue
```

and functions inside the class.

You learned about:

- class
- object
- private data
- public methods
- encapsulation
- constructors
- default constructor
- parameterized constructor
- `this->`

- `new[]`
- destructor
- `delete[]`
- scope resolution operator `::`
- object syntax like `q.enqueue(10)`

The most important Phase 4 idea is:

The queue algorithm is mostly the same, but C++ OOP makes the program cleaner, safer, and better organized.

55. Final Takeaway

Phase 4 is not about learning a completely new queue.

It is about writing the same queue in a better C++ style.

Remember:

Same FIFO logic.
Same front and rear idea.
Same enqueue and dequeue behavior.
Better organization using class and object.

So the final beginner-friendly takeaway is:

A C++ OOP queue keeps its data private and provides public functions like enqueue, dequeue, and display so the queue can be used safely and cleanly.